CO3

ASSESSMENT TOOL-2

NAME : P. Ranjithkumar **DATE:04.08.2026**

REG.NO: 192411119 **Tuesday**

COURSE CODE : CSA1103

COURSE NAME : OOAD

1. Online Hotel Reservation System

Analysis

The Online Hotel Reservation System allows customers to search hotels, book rooms, make payments, and submit reviews. Hotel managers update room availability and prices, while administrators manage users and transactions.

Object-Oriented Principles

Encapsulation

- Data and methods are grouped into classes.
- Example:
  - `Customer` → customerID, name, email
  - Methods: searchHotel(), bookRoom(), makePayment()

Inheritance

- A common class **User** is created.
- Customer, HotelManager, and Administrator inherit from User.

```
        User
       / |  \
Customer Manager Admin
```

Polymorphism

- Different users perform the same function differently.
- Example:
  - login() behaves differently for Customer, Manager, and Admin.

Major Classes

- User
- Customer
- Hotel
- Room
- Booking
- Payment
- Review
- HotelManager
- Administrator

Suitable SDLC Model

Agile Model

Justification

- Requirements change frequently.
- Easy to add new features.
- Continuous testing and customer feedback.
- Faster development.

Conclusion

Object-oriented principles improve modularity and code reuse, while Agile provides flexibility and quick delivery.

2. University Management System

Analysis

The system manages admissions, course registration, examinations, and result processing.

Major Classes

Student

- studentID
- name

- registerCourse()

Faculty

- facultyID
- uploadMarks()

Course

- courseID
- courseName

Examination

- examID
- conductExam()

Result

- marks
- grade

Administrator

- manageUsers()

Relationships

```
Student -------- Enrolls -------- Course

Faculty -------- Teaches -------- Course

Course -------- Conducts -------- Examination

Examination ----- Generates ----- Result
```

OOP Concepts

Encapsulation

- Student details stored privately.

Inheritance

```
        Person
      /    |     \
 Student Faculty Admin
```

Polymorphism

- login() works differently for Student, Faculty, and Admin.

Suitable SDLC Model

Incremental Model

Justification

- Modules developed separately.
- Easy maintenance.
- New modules added later.
- Reduced development risk.

Conclusion

Incremental development allows smooth implementation and future expansion.

3. Online Retail Shopping Platform

Analysis

Customers browse products, place orders, track deliveries, and make payments. Vendors manage products, and administrators manage inventory.

Modular Design

Module 1

- Customer Management

Module 2

- Product Management

Module 3

- Order Management

Module 4

- Payment Module

Module 5

- Delivery Tracking

Module 6

- Inventory Management

OOP Principles

Encapsulation

- Product information stored in Product class.

Inheritance

```
        User
      /     \
Customer Vendor
         |
    Administrator
```

Polymorphism

- Payment can be made through:
    - UPI
    - Credit Card
    - Net Banking

All implement the same method:

`makePayment()`

SDLC Models

- Waterfall
- Spiral
- Agile
- Incremental

Best Model

Agile Model

Justification

- Frequent updates
- Easy feature addition
- Customer feedback
- Quick releases

Conclusion

Object-oriented design creates reusable modules, and Agile ensures continuous improvement.

4. Smart Waste Management System

Analysis

IoT-enabled bins monitor waste levels and notify collection vehicles. Citizens report waste problems using a mobile application.

Major Classes

- SmartBin
- Sensor
- Vehicle
- Citizen
- WasteReport
- Administrator

OOP Principles

Encapsulation

- Waste level stored inside SmartBin.

Inheritance

```
          User
         /    \
 Citizen Administrator
```

Polymorphism

```
sendNotification()
```

```
SmartBin
CitizenApp

Both send notifications differently.
```

Suitable SDLC Model

Spiral Model

Justification

- IoT integration
- Sensor testing
- Risk management
- Requirement changes

Conclusion

Spiral Model is suitable because it supports continuous testing and risk analysis.

5. Smart Healthcare Management System

Analysis

Patients book appointments, doctors update diagnoses, and administrators manage billing and staff.

OOP Principles

Encapsulation

Patient Class

- patientID
- name
- medicalHistory

Methods

- bookAppointment()
- viewRecord()

Inheritance

```
            User
          /    |      \
     Patient Doctor Admin
```

Polymorphism

```
login()

Patient Login

Doctor Login

Admin Login
```

Major Classes

- Patient
- Doctor
- Appointment
- MedicalRecord
- Prescription
- Payment
- Department
- Administrator

Suitable SDLC Model

Agile Model

Justification

- Frequent requirement changes
- Secure development
- Continuous testing
- Easy integration

Conclusion

Agile ensures reliable healthcare software with continuous improvement.

6. Smart Library Management System

Analysis

Students and faculty search books, borrow, reserve, return books, and pay fines.

Major Classes

Book

Attributes

- bookID
- title
- author

Methods

- searchBook()

Member

Attributes

- memberID
- name

Methods

- borrowBook()
- returnBook()

Librarian

Methods

- addBook()
- removeBook()

Fine

Methods

- calculateFine()

Administrator

Methods

- generateReport()

Relationships

```
Member ------- Borrows ------- Book

Librarian ----- Manages ------ Book

Administrator --- Generates --- Reports

Member -------- Pays -------- Fine
```

OOP Concepts

Encapsulation

Book information is stored securely inside the Book class.

Inheritance

```
            Person
          /      |      \
 Student Faculty Librarian
```

Polymorphism

```
borrowBook()

Student

Faculty

Different borrowing limits.
```

Suitable SDLC Model

Incremental Model

Justification

- Library modules can be developed separately.
- Easy maintenance.
- Future features can be added.
- Faster implementation.

Conclusion

Incremental development with object-oriented principles provides a flexible, maintainable, and efficient library management system.